ROS 2 URDF

Robot Description Format

Complete Tutorial

Building a Differential Drive Robot from Scratch

What you will build:

A complete URDF description of a two-wheeled differential drive robot with a lidar sensor — visualised in RViz2 with a launch file that opens everything automatically.

Parts: chassis, left wheel, right wheel, caster wheel, lidar sensor

New package: `my_robot_description`

Contains: `urdf/` , `launch/` , `config/`
No Python nodes — this is a description-only package.

Environment

Item	Value
ROS 2	Humble Hawksbill
OS	Ubuntu 22.04 LTS
Visualiser	RViz2
Description format	URDF (XML)
Robot type	Differential drive

Contents

1 What is URDF and Why Does It Matter? 3
1.1 What URDF Enables . . . 3
1.2 URDF in the ROS 2 Learning Path . . . 4

2 The Two Building Blocks: Links and Joints 4
2.1 Links — Rigid Body Parts . . . 4
2.2 Joints — Connections Between Links . . . 5
2.3 The Link-Joint Tree . . . 5

3 Geometry Types 5

4 The Robot We Are Building 6

5 Create the Package 6
5.1 Create the package . . . 6
5.2 Create the folder structure . . . 6
5.3 Final package structure . . . 7

6 Update package.xml 7

7 Update setup.py 7

8 Install Required Tools 8

9 Write the URDF File 9
9.1 Root element and materials . . . 9
9.2 Dummy root link . . . 9
9.3 Chassis (base_link) . . . 10
9.4 Left Wheel . . . 10
9.5 Right Wheel . . . 11
9.6 Caster Wheel . . . 12
9.7 Lidar Sensor . . . 12

10 Write the Launch File 13

11 Build the Package 15

12 First Launch and RViz2 Setup 15
12.1 First launch — before saving config . . . 15
12.2 Set up RViz2 manually (first time only) . . . 16
12.3 Save the RViz2 config . . . 16
12.4 Rebuild to install the config file . . . 16
12.5 From now on — automatic . . . 16

13 What You Should See in RViz2 17

14 URDF Coordinate System and origin 17
14.1 origin explained . . . 17

14.2 Our robot's joint positions 18

15 Verify with CLI Tools **18**

16 Complete URDF File **18**

17 Common Mistakes and Fixes **21**

18 Quick Reference **21**

18.1 Commands 22

18.2 URDF Element Summary 22

18.3 Robot Parts Summary 22

1 What is URDF and Why Does It Matter?

Everything you have built so far — publishers, subscribers, services, actions, launch files, parameters — deals with **communication and logic**. ROS 2 knows how your nodes talk to each other, but it has absolutely no idea what your robot physically looks like. It does not know how big it is, how many wheels it has, where its sensors are mounted, or how its parts connect and move relative to each other.

URDF — the Unified Robot Description Format — solves this. It is an XML file that gives your robot a complete physical identity inside ROS 2. Once a URDF exists, every tool in the ROS 2 ecosystem can use it. RViz2 renders the robot in 3D. Gazebo simulates it with realistic physics. The navigation stack knows the robot's footprint for collision avoidance. The arm planning library knows the reach and joint limits. The transform system knows exactly where every part of the robot is at every moment. All of this flows from a single URDF file.

URDF is not just a beginner concept. Every real robot — TurtleBot, Spot, the Universal Robots arm, NASA's humanoids — is described in URDF or its extension XACRO. When you download an open-source robot package, the first file you look at is almost always the URDF. When you integrate a new sensor, you add it to the URDF. When simulation behaves strangely, you debug the URDF. It is the single most important file in any robot project.

1.1 What URDF Enables

Tool	What it uses URDF for
RViz2	Renders the 3D robot model, shows coordinate frames
Gazebo	Simulates physics, collision, sensors, actuators
TF2	Computes transforms between every link at every moment
Nav2	Uses robot footprint for obstacle avoidance
MoveIt2	Plans arm trajectories using joint definitions
ros2_control	Maps joints to hardware interfaces

1.2 URDF in the ROS 2 Learning Path

```
Publisher / Subscriber / Services / Actions / Launch / Parameters
                                |
                                v
                        URDF  <-- You are here
                                |
                                v
                             RViz2
                                |
                                v
                              TF2
                                |
                                v
                             Gazebo
                                |
                                v
                          ros2_control
                                |
                                v
                               Nav2
```

None of the later steps work without URDF as their foundation. A robot without a URDF cannot be visualised, simulated, or autonomously navigated.

2 The Two Building Blocks: Links and Joints

Every URDF is constructed from exactly two elements. Understanding these two elements is the entire mental model you need.

2.1 Links — Rigid Body Parts

A **link** is any rigid, non-deformable part of the robot. The chassis is a link. Each wheel is a link. The lidar sensor is a link. Every part that moves as one solid piece is one link.

Each link has three sub-elements:

Sub-element	Purpose
`visual`	What the link looks like in RViz2. Shape and colour.
`collision`	The simplified shape used for collision detection in Gazebo. Often the same as visual but can be simpler for performance.
`inertial`	Mass and moment of inertia. Needed for physics simulation. Not needed for RViz2 visualisation only.

2.2 Joints — Connections Between Links

A **joint** defines how two links connect and how they can move relative to each other. Every joint has a parent link and a child link. The parent is the link higher up in the tree (closer to the root). The child is the link attached to it.

Joint Type	Movement	Example
`fixed`	None — completely rigid	Camera on chassis, lidar mount
`continuous`	Rotates endlessly	Wheel (spins forever)
`revolute`	Rotates within angle limits	Robot arm joint
`prismatic`	Slides linearly with limits	Linear actuator
`floating`	6 degrees of freedom	Rarely used in practice

2.3 The Link-Joint Tree

URDF describes the robot as a **tree structure**. There is one root link. Every other link is a child, connected through a joint. The tree for our differential drive robot looks like this:

```
base_footprint  (root -- dummy link)
      |
      | base_footprint_joint (fixed)
      |
  base_link  (chassis -- blue box)
      |
      |-- left_wheel_joint (continuous)
      |         |
      |     left_wheel  (black cylinder)
      |
      |-- right_wheel_joint (continuous)
      |         |
      |     right_wheel  (black cylinder)
      |
      |-- caster_wheel_joint (fixed)
      |         |
      |     caster_wheel  (grey sphere)
      |
      |-- lidar_joint (fixed)
                |
            lidar_link  (red cylinder)
```

3 Geometry Types

URDF supports four built-in geometry shapes. All dimensions are in **metres**.

Shape	XML tag	Parameters	Used for
Box	`<box/>`	`size="length width height"`	Chassis, walls, shelves
Cylinder	`<cylinder/>`	`radius="r" length="l"`	Wheels, poles, lidar
Sphere	`<sphere/>`	`radius="r"`	Caster wheel, ball joint
Mesh	`<mesh/>`	`filename="model.stl"`	Realistic 3D models

4 The Robot We Are Building

A **differential drive robot** is the most common beginner robot and the foundation of most commercial mobile robots. It has two independently driven wheels on the same axis. By driving them at different speeds the robot turns. A passive caster wheel at the front prevents tipping. A lidar sensor on top scans the environment.

```
          [red cylinder]      lidar sensor on top
         [  blue box  ]       chassis (0.4 x 0.3 x 0.1 m)
        /               \
  [black cyl]        [black cyl]   left and right wheels
        \               /          radius 0.1 m, width 0.05 m
         [grey sphere]             front caster wheel
                                   radius 0.05 m
```

Part	Link Name	Shape	Dimensions	Joint Type
Chassis	base_link	Box	0.4 x 0.3 x 0.1 m	fixed to base_footprint
Left wheel	left_wheel	Cylinder	r=0.1m, l=0.05m	continuous
Right wheel	right_wheel	Cylinder	r=0.1m, l=0.05m	continuous
Caster	caster_wheel	Sphere	r=0.05m	fixed
Lidar	lidar_link	Cylinder	r=0.05m, l=0.04m	fixed

5 Create the Package

5.1 Create the package

```
cd ~/ros2_ws/src

ros2 pkg create my_robot_description \
  --build-type ament_python \
  --dependencies rclpy
```

5.2 Create the folder structure

```
cd my_robot_description
mkdir urdf
mkdir launch
mkdir config
touch urdf/my_robot.urdf
touch launch/display.launch.py
```

5.3 Final package structure

```
my_robot_description/
 urdf/
    my_robot.urdf          <- robot URDF description
 launch/
    display.launch.py      <- opens RViz2 automatically
 config/
    rviz_config.rviz       <- saved RViz2 settings (created later)
 my_robot_description/
    __init__.py
 package.xml
 setup.cfg
 setup.py
```

6 Update package.xml

Listing 1: package.xml

```xml
<?xml version="1.0"?>
<package format="3">
  <name>my_robot_description</name>
  <version>0.0.0</version>
  <description>My robot URDF description</description>
  <maintainer email="you@email.com">your_name</maintainer>
  <license>Apache-2.0</license>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>joint_state_publisher_gui</exec_depend>
  <exec_depend>rviz2</exec_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

7 Update setup.py

Listing 2: setup.py

```python
import os
from glob import glob
from setuptools import find_packages, setup

package_name = 'my_robot_description'

setup(
    name=package_name,
```

```
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),

        # Install URDF files
        (os.path.join('share', package_name, 'urdf'),
            glob('urdf/*.urdf')),

        # Install launch files
        (os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py')),

        # Install RViz2 config files
        (os.path.join('share', package_name, 'config'),
            glob('config/*.rviz')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='your_name',
    maintainer_email='you@email.com',
    description='My robot URDF description',
    license='Apache-2.0',
    extras_require={'test': ['pytest']},
    entry_points={
        'console_scripts': [],
    },
)
```

8 Install Required Tools

```
sudo apt install ros-humble-joint-state-publisher-gui
sudo apt install ros-humble-robot-state-publisher
sudo apt install ros-humble-rviz2
```

Tool	Purpose
`robot_state_publisher`	Reads the URDF and publishes TF transforms for every link. Without this, RViz2 has no idea where each link is.
`joint_state_publisher_gui`	Provides a GUI with sliders to manually move the robot's joints. Lets you spin the wheels interactively in RViz2.
`rviz2`	The 3D visualiser. Renders the robot model, coordinate frames, sensor data, and paths.

9 Write the URDF File

Open `urdf/my_robot.urdf` and write the following. The file is built section by section — read the comments carefully as they explain every value.

No Special Characters in Comments

Avoid using special characters like `=`, `*`, or unicode symbols inside XML comments in URDF files. Some versions of the ROS 2 XML parser reject them and produce an "invalid robot_description" error. Keep comments simple plain text.

9.1 Root element and materials

Listing 3: my_robot.urdf — Part 1: root and materials

```xml
<?xml version="1.0"?>
<robot name="my_robot">

  <!-- Colors used by visual elements in RViz2 -->
  <!-- rgba: red green blue alpha, values 0.0 to 1.0 -->

  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>

  <material name="black">
    <color rgba="0.0 0.0 0.0 1.0"/>
  </material>

  <material name="grey">
    <color rgba="0.5 0.5 0.5 1.0"/>
  </material>

  <material name="red">
    <color rgba="0.8 0.0 0.0 1.0"/>
  </material>
```

9.2 Dummy root link

Listing 4: my_robot.urdf — Part 2: base_footprint

```xml
  <!-- base_footprint is a dummy root link with no geometry.
       It fixes a KDL warning: KDL does not support inertia
       on the root link. By making base_footprint the root
       and giving it no inertia, the warning disappears.
       base_link becomes a child connected by a fixed joint. -->

  <link name="base_footprint"/>

  <joint name="base_footprint_joint" type="fixed">
    <parent link="base_footprint"/>
```

```
    <child  link="base_link"/>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
  </joint>
```

Why base_footprint?

`base_footprint` is a ROS convention. It represents the robot's position projected onto the ground plane — useful for navigation. `base_link` is the physical centre of the robot. Most real robots have both.

9.3 Chassis (base_link)

Listing 5: my_robot.urdf — Part 3: chassis

```
  <!-- Chassis: the main body of the robot.
       Shape: box 0.4m long, 0.3m wide, 0.1m tall.
       All other links connect to this link. -->

  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.4 0.3 0.1"/>
      </geometry>
      <material name="blue"/>
    </visual>
    <collision>
      <geometry>
        <box size="0.4 0.3 0.1"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="5.0"/>
      <inertia ixx="0.1" ixy="0.0" ixz="0.0"
               iyy="0.1" iyz="0.0"
               izz="0.1"/>
    </inertial>
  </link>
```

9.4 Left Wheel

Listing 6: my_robot.urdf — Part 4: left wheel

```
  <!-- Left wheel: cylinder, radius 0.1m, width 0.05m.
       Joint type continuous means it spins without limit.
       origin xyz: placed at left side (y=+0.175), ground level (
          z=-0.05).
       origin rpy: rotated -90 degrees around X to face sideways.
       axis xyz="0 0 1": spins around its own Z axis. -->

  <link name="left_wheel">
    <visual>
```

```
      <geometry>
        <cylinder radius="0.1" length="0.05"/>
      </geometry>
      <material name="black"/>
    </visual>
    <collision>
      <geometry>
        <cylinder radius="0.1" length="0.05"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.01" ixy="0.0" ixz="0.0"
               iyy="0.01" iyz="0.0"
               izz="0.01"/>
    </inertial>
  </link>

  <joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child  link="left_wheel"/>
    <origin xyz="0.0 0.175 -0.05" rpy="-1.5708 0 0"/>
    <axis xyz="0 0 1"/>
  </joint>
```

9.5 Right Wheel

Listing 7: my_robot.urdf — Part 5: right wheel

```
  <!-- Right wheel: mirror of left wheel.
       origin y=-0.175 places it on the right side. -->

  <link name="right_wheel">
    <visual>
      <geometry>
        <cylinder radius="0.1" length="0.05"/>
      </geometry>
      <material name="black"/>
    </visual>
    <collision>
      <geometry>
        <cylinder radius="0.1" length="0.05"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.01" ixy="0.0" ixz="0.0"
               iyy="0.01" iyz="0.0"
               izz="0.01"/>
    </inertial>
  </link>
```

```

<joint name="right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child  link="right_wheel"/>
  <origin xyz="0.0 -0.175 -0.05" rpy="-1.5708 0 0"/>
  <axis xyz="0 0 1"/>
</joint>
```

9.6 Caster Wheel

Listing 8: my_robot.urdf — Part 6: caster wheel

```
<!-- Caster wheel: passive front support sphere.
     Fixed joint: it does not spin independently in URDF.
     Gazebo handles caster physics separately.
     Positioned at front (x=+0.15) and low (z=-0.1). -->

<link name="caster_wheel">
  <visual>
    <geometry>
      <sphere radius="0.05"/>
    </geometry>
    <material name="grey"/>
  </visual>
  <collision>
    <geometry>
      <sphere radius="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.1"/>
    <inertia ixx="0.001" ixy="0.0" ixz="0.0"
             iyy="0.001" iyz="0.0"
             izz="0.001"/>
  </inertial>
</link>

<joint name="caster_wheel_joint" type="fixed">
  <parent link="base_link"/>
  <child  link="caster_wheel"/>
  <origin xyz="0.15 0.0 -0.1" rpy="0 0 0"/>
</joint>
```

9.7 Lidar Sensor

Listing 9: my_robot.urdf — Part 7: lidar and closing tag

```
<!-- Lidar: mounted on top of chassis.
     Fixed joint: moves with the robot as one rigid body.
     Positioned at centre (x=0, y=0) and on top (z=+0.07). -->

```

```
  <link name="lidar_link">
    <visual>
      <geometry>
        <cylinder radius="0.05" length="0.04"/>
      </geometry>
      <material name="red"/>
    </visual>
    <collision>
      <geometry>
        <cylinder radius="0.05" length="0.04"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.001" ixy="0.0" ixz="0.0"
               iyy="0.001" iyz="0.0"
               izz="0.001"/>
    </inertial>
  </link>

  <joint name="lidar_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="lidar_link"/>
    <origin xyz="0.0 0.0 0.07" rpy="0 0 0"/>
  </joint>

</robot>
```

10 Write the Launch File

This launch file starts three nodes and automatically loads the saved RViz2 config so the robot appears immediately without any manual setup.

Listing 10: launch/display.launch.py

```
import os
from ament_index_python.packages import
   get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():

    # Resolve path to the URDF file in the installed package
    urdf_file = os.path.join(
        get_package_share_directory('my_robot_description'),
        'urdf',
        'my_robot.urdf'
    )

```

```python
    # Read the entire URDF into a string
    with open(urdf_file, 'r') as f:
        robot_description = f.read()

    # Resolve path to the saved RViz2 config file
    rviz_config_file = os.path.join(
        get_package_share_directory('my_robot_description'),
        'config',
        'rviz_config.rviz'
    )

    # robot_state_publisher:
    # Reads the URDF and publishes TF transforms for all links.
    # Publishes on /robot_description topic.
    robot_state_publisher_node = Node(
        package    = 'robot_state_publisher',
        executable = 'robot_state_publisher',
        name       = 'robot_state_publisher',
        output     = 'screen',
        parameters = [{'robot_description': robot_description}],
    )

    # joint_state_publisher_gui:
    # Provides GUI sliders to manually move joints.
    # Publishes joint positions on /joint_states topic.
    joint_state_publisher_gui_node = Node(
        package    = 'joint_state_publisher_gui',
        executable = 'joint_state_publisher_gui',
        name       = 'joint_state_publisher_gui',
        output     = 'screen',
    )

    # rviz2:
    # 3D visualiser. Opens with saved config automatically.
    # -d flag loads the config file on startup.
    rviz_node = Node(
        package    = 'rviz2',
        executable = 'rviz2',
        name       = 'rviz2',
        output     = 'screen',
        arguments  = ['-d', rviz_config_file],
    )

    return LaunchDescription([
        robot_state_publisher_node,
        joint_state_publisher_gui_node,
        rviz_node,
    ])
```

How the Three Nodes Work Together

```
my_robot.urdf
      |
      | (read by launch file)
      v
robot_state_publisher
      |
      | publishes /robot_description (URDF string)
      | publishes /tf (transforms for fixed joints)
      v
joint_state_publisher_gui
      |
      | GUI sliders let you move joints
      | publishes /joint_states (wheel angles)
      v
robot_state_publisher
      |
      | listens to /joint_states
      | publishes /tf (transforms for moving joints)
      v
rviz2
      |
      | subscribes to /robot_description and /tf
      | renders 3D robot model
```

11 Build the Package

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

Expected Output

```
Starting >>> my_robot_description
Finished <<< my_robot_description [2.1s]
Summary: 1 package finished [2.5s]
```

12 First Launch and RViz2 Setup

12.1 First launch — before saving config

On the first launch, no saved config exists yet so RViz2 opens to a blank view. You must set it up manually once.

```
ros2 launch my_robot_description display.launch.py
```

12.2 Set up RViz2 manually (first time only)

Follow these steps in the RViz2 window:

1. In the left **Displays** panel, find **Global Options**
2. Change **Fixed Frame** from `map` to `base_footprint`
3. Click **Add** at the bottom of the Displays panel
4. Select **RobotModel** and click **OK**
5. The robot should appear in the 3D view
6. Scroll your mouse wheel to zoom in if needed
7. Left-click and drag to rotate the view

12.3 Save the RViz2 config

Once the robot is visible and the view looks good:

1. In RViz2 top menu: **File** → **Save Config As**
2. Navigate to: `~/ros2_ws/src/my_robot_description/config/`
3. Save as: `rviz_config.rviz`

12.4 Rebuild to install the config file

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

12.5 From now on — automatic

Every subsequent launch opens RViz2 with the robot already visible:

```
ros2 launch my_robot_description display.launch.py
```

No manual setup required.

13 What You Should See in RViz2

```
Left panel (Displays):
  Global Options
    Fixed Frame: base_footprint
    Global Status: Ok
  Grid            (checkmark)
  RobotModel      (checkmark)
    Status: Ok

3D viewport:
  Blue box            <- chassis (base_link)
  Two black cylinders <- left and right wheels
  Grey sphere         <- caster wheel at front
  Red cylinder        <- lidar on top

Joint State Publisher GUI window:
  Slider: left_wheel_joint   <- spin left wheel
  Slider: right_wheel_joint  <- spin right wheel
```

Moving the sliders in the GUI window rotates the corresponding wheels in the 3D view in real time.

14 URDF Coordinate System and origin

Every joint has an `<origin>` that positions the child link relative to the parent link. Understanding the coordinate system is essential for placing parts correctly.

Axis	Direction	Meaning
X	Forward	Positive X points to the front of the robot
Y	Left	Positive Y points to the left side
Z	Up	Positive Z points upward

14.1 origin explained

```
<origin xyz="0.0 0.175 -0.05" rpy="-1.5708 0 0"/>
          |   |       |         |
          |   |       |         rpy = roll pitch yaw (radians)
          |   |       |         -1.5708 = -90 degrees around X axis
          |   |       |         (rotates cylinder to face sideways)
          |   |
          |   y = +0.175  -> left side of robot
          |   z = -0.05   -> below the chassis centre
          |
          x = 0.0 -> same forward position as chassis centre
```

14.2 Our robot's joint positions

Joint	Origin explanation
`base_footprint_joint`	`xyz="0 0 0.1"` — base_link sits 0.1m above the ground plane
`left_wheel_joint`	`xyz="0 0.175 -0.05"` — left side, slightly below chassis centre
`right_wheel_joint`	`xyz="0 -0.175 -0.05"` — right side, slightly below chassis centre
`caster_wheel_joint`	`xyz="0.15 0 -0.1"` — front of robot, at ground level
`lidar_joint`	`xyz="0 0 0.07"` — top centre of chassis

15 Verify with CLI Tools

While the launch is running, open a second terminal and run these:

```
# Check robot_description topic is being published
source ~/ros2_ws/install/setup.bash
ros2 topic list

# Verify URDF content is published correctly
ros2 topic echo /robot_description --once

# Check joint states are being published
ros2 topic echo /joint_states

# View the TF tree
ros2 run tf2_tools view_frames

# Check all active nodes
ros2 node list
```

Expected topic list

```
/joint_states
/parameter_events
/robot_description
/rosout
/tf
/tf_static
```

16 Complete URDF File

For reference, here is the complete `my_robot.urdf` in one block:

Listing 11: urdf/my_robot.urdf — complete file

```
<?xml version="1.0"?>
```

```
<robot name="my_robot">

  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>
  <material name="black">
    <color rgba="0.0 0.0 0.0 1.0"/>
  </material>
  <material name="grey">
    <color rgba="0.5 0.5 0.5 1.0"/>
  </material>
  <material name="red">
    <color rgba="0.8 0.0 0.0 1.0"/>
  </material>

  <link name="base_footprint"/>

  <joint name="base_footprint_joint" type="fixed">
    <parent link="base_footprint"/>
    <child  link="base_link"/>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
  </joint>

  <link name="base_link">
    <visual>
      <geometry><box size="0.4 0.3 0.1"/></geometry>
      <material name="blue"/>
    </visual>
    <collision>
      <geometry><box size="0.4 0.3 0.1"/></geometry>
    </collision>
    <inertial>
      <mass value="5.0"/>
      <inertia ixx="0.1" ixy="0.0" ixz="0.0"
               iyy="0.1" iyz="0.0" izz="0.1"/>
    </inertial>
  </link>

  <link name="left_wheel">
    <visual>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
      <material name="black"/>
    </visual>
    <collision>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.01" ixy="0.0" ixz="0.0"
               iyy="0.01" iyz="0.0" izz="0.01"/>
    </inertial>
```

```
  </link>
  <joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child  link="left_wheel"/>
    <origin xyz="0.0 0.175 -0.05" rpy="-1.5708 0 0"/>
    <axis xyz="0 0 1"/>
  </joint>

  <link name="right_wheel">
    <visual>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
      <material name="black"/>
    </visual>
    <collision>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.01" ixy="0.0" ixz="0.0"
               iyy="0.01" iyz="0.0" izz="0.01"/>
    </inertial>
  </link>
  <joint name="right_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child  link="right_wheel"/>
    <origin xyz="0.0 -0.175 -0.05" rpy="-1.5708 0 0"/>
    <axis xyz="0 0 1"/>
  </joint>

  <link name="caster_wheel">
    <visual>
      <geometry><sphere radius="0.05"/></geometry>
      <material name="grey"/>
    </visual>
    <collision>
      <geometry><sphere radius="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.001" ixy="0.0" ixz="0.0"
               iyy="0.001" iyz="0.0" izz="0.001"/>
    </inertial>
  </link>
  <joint name="caster_wheel_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="caster_wheel"/>
    <origin xyz="0.15 0.0 -0.1" rpy="0 0 0"/>
  </joint>

  <link name="lidar_link">
    <visual>
```

```
      <geometry><cylinder radius="0.05" length="0.04"/></geometry>
      <material name="red"/>
    </visual>
    <collision>
      <geometry><cylinder radius="0.05" length="0.04"/></geometry>
    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.001" ixy="0.0" ixz="0.0"
               iyy="0.001" iyz="0.0" izz="0.001"/>
    </inertial>
  </link>
  <joint name="lidar_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="lidar_link"/>
    <origin xyz="0.0 0.0 0.07" rpy="0 0 0"/>
  </joint>

</robot>
```

17 Common Mistakes and Fixes

Mistake	Error	Fix
Special characters in comments	`Invalid robot_description`	Use plain text in XML comments only
Inertia on root link	KDL warning about root link inertia	Add `base_footprint` dummy link as root
Forgot `<axis>` in continuous joint	Wheel does not spin correctly	Add `<axis xyz="0 0 1"/>` to all wheel joints
Wrong `rpy` on wheel joint	Wheel faces wrong direction	Use `rpy="-1.5708 0 0"` to rotate cylinder sideways
Forgot to add `urdf/` to `data_files`	URDF file not found at launch	Add `glob('urdf/*.urdf')` to `setup.py`
Fixed Frame set to `map` in RViz2	Robot not visible, transform error	Change Fixed Frame to `base_footprint`
Forgot to rebuild after saving `rviz_config.rviz`	RViz2 opens blank again	`colcon build` then `source install/setup.bash`
Missing `robot_state_publisher` dependency	`Package not found`	`sudo apt install ros-humble-robot-state-pub`

18 Quick Reference

18.1 Commands

```
# Build
colcon build --packages-select my_robot_description
source install/setup.bash

# Launch robot in RViz2
ros2 launch my_robot_description display.launch.py

# Verify URDF is published
ros2 topic echo /robot_description --once

# View the TF tree
ros2 run tf2_tools view_frames

# List all active nodes
ros2 node list
```

18.2 URDF Element Summary

Element	Purpose
`<robot name="">`	Root element, wraps everything
`<material>`	Defines a named colour for reuse
`<link name="">`	A rigid body part of the robot
`<visual>`	What the link looks like in RViz2
`<collision>`	Collision geometry used by Gazebo
`<inertial>`	Mass and inertia for physics simulation
`<joint name="" type="">`	Connection between two links
`<parent link="">`	The parent link (higher in tree)
`<child link="">`	The child link (attached to parent)
`<origin xyz="" rpy="">`	Position and rotation offset
`<axis xyz="">`	Axis of rotation for moving joints

18.3 Robot Parts Summary

Part	Link	Shape	Joint	Colour
Root	base_footprint	none	—	—
Chassis	base_link	box	fixed	blue
L. Wheel	left_wheel	cylinder	continuous	black
R. Wheel	right_wheel	cylinder	continuous	black
Caster	caster_wheel	sphere	fixed	grey
Lidar	lidar_link	cylinder	fixed	red

End of Tutorial — ROS 2 URDF: Robot Description Format